STAT 4710/5710: Homework 2

Ying Jin

Due: September 30, 2025 at 11:59am

Contents

Instructions

Materials and collaboration

The policy on allowed materials and collaboration is as stated on the Syllabus:

> "Students are permitted to work together on homework assignments, but must write up and submit solutions individually. In particular, students may not copy each others' solutions. Students may consult all course materials, textbooks, the internet, or AI tools (e.g. ChatGPT) to complete their homework. Students may not use solutions to problems that may be available online and/or from past iterations of the course. For each homework, students must disclose all classmates with whom they collaborated, which AI tools they used, and how they used them. Failure to do so will result in a 10-point penalty."

In accordance with this policy,

Please disclose all classmates with whom you collaborated:

Please disclose which AI tools you used, and how you used them:

Failure to answer the above questions will result in a 10-point penalty.

Writeup

Use `homework-2-template.Rmd` as a starting point for your writeup, adding your solutions after "**Solution**". Add your R code using code chunks and add your text answers using **bold text**. Consult the preparing reports guide for guidance on compilation, creation of figures and tables, and presentation quality. If the instructions ask you to "print a tibble", you should print the tibble directly.

Programming

We encourage you to follow the `tidyverse` paradigm for data visualization, manipulation, and wrangling, as in the first homework, to practice tidyverse workflows (e.g., `filter`, `mutate`, `group_by`, `summarize`, `join`, `ggplot2`).

Students who strongly prefer to use Python for homework may do so. If you choose this option, you are **required** to present both code chunks and the corresponding outputs/interpretations in your PDF. In addition, it's recommended that you:

- Use `pandas` for data manipulation (preferred, since it parallels `dplyr`).
- Use `plotnine` for visualization (preferred, since it parallels ggplot2). If using seaborn/matplotlib, you must still include the required plot elements (labels, 45° line, smoothing curve, facets, etc.).

```r
library(tidyverse)
library(splines)
```

Grading

The point value for each problem sub-part is indicated. There are 100 points possible on this homework, evaluated based on the correctness of the numerical results and plots. When a problem asks for a plot:

- **Axes and labels**: Always include informative axis labels and, if appropriate, titles.
- **Required elements**: If the instructions ask for a 45° line, smoothing curve, or facets, these must be included.
- **Clarity**: Points may be deducted if plots are cluttered, unreadable, or missing labels.

Submission

Compile your writeup to PDF and submit to Gradescope.

1 Case study: spline regression model selection (64 points)

In this case study, you will use a semi-synthetic data to model hourly wage as a function of age using natural splines, tuning the spline's degrees of freedom (df) two ways:

- a validation set approach that you implement, and
- 10-fold cross-validation using the provided helper `cross_validate_spline()`.

You will repeat the full workflow separately for two education levels: "HS Grad" and "College Grad".

1.1 Import data (2 points)

Import the data `wages.csv` into a tibble called `wage_df`, replace the `wage` column by its logarithm value, and print the imported tibble. The log wage in the `wage` column will be our response variable to predict.

1.2 Tidy (2 points)

- (2 points) In `wage_df`, keep only the following columns needed for this problem and drop all rows with any NA values in these columns:
  - Outcome: `wage` (continuous, $/hr)
  - Predictor for spline: `age` (numeric)
  - Grouping variable: `education` (factor)
- (2 points) In the study, we will analyze only two education levels: `HS Grad` and `College Grad`. Filter the data to keep only these two levels. Print the resulting tibble.

1.3 Explore (8 points)

- (2 points) How many `HS Grad` people are there in the dataset? How many `College Grad` people? Write code to figure these out.
- (2 points) Produce plots to compare the distributions of `wage` between `HS Grad` and `College Grad`. Write code to produce these plots.
- (2 points) Produce plots to visualize the relationship between `wage` and `age` separately for `HS Grad` and `College Grad`. Write code to produce these plots.
- (2 points) Did you notice any patterns in the plots?

1.4 Prepare for modeling (1 point)

You'll fit natural spline regressions of the form `wage ~ ns(age, df = df)` where we'll tune the complexity via `df` values in $\{2, 3, \ldots, 20\}$.

- Below is an incomplete code to split `wage_df` into training (80%) and test (20%) sets. Please continue to complete the code for data splitting, using the rows in `train_samples` below for training and the remaining for test. Store these in tibbles called `wage_train` and `wage_test`, respectively.

```r
set.seed(4710) # set seed for reproducibility
n <- nrow(wage_df)
train_samples <- sample(1:n, size = 0.8 * n, replace = FALSE)
```

1.5 Validation set approach to model selection (33 points)

1.5.1 Data splitting (3 points)

- (1 point) Since we will work with two education groups, we will fit individual models on the two groups. Separate `wage_train` into `wage_train_HS` and `wage_train_coll`. Likewise for `wage_test`.
- (2 points) Mimic the code for splitting the training/test sample in Section 4.1 to do the following: split each group's training set into a learning (70%) set and a validation (30%) set, naming them as

`wage_learn_HS`, `wage_val_HS`, `wage_learn_coll`, and `wage_val_coll`. Please set the random seed via `set.seed(4710)`.

1.5.2 Estimate MSE given a df (15 points)

- (10 points) Using the incomplete template below to write a function called `validation_mse()` that takes as input a learning set tibble `learn_data`, a validation set tibble `val_data`, and a `df` value, and does the following:
  - Fit a spline model by `lm(wage ~ ns(age, df = ...), data = ...` on `learn_data`.
  - Make predictions on the validation data `val_data`.
  - Computes and returns the MSE on the validation data `val_data`.
- (5 points) Using the function `validation_mse()`, compute the validation MSE for each `df` value in {2, 3, . . . , 20} separately for the two education groups. Store the results in tibbles called `val_errors_HS` and `val_errors_coll`, each with two columns named `df` and `val_mse`. Print these tibbles.

```
validation_mse <- function(learn_data, val_data, df) {
  spline_fit <- [TO COMPLETE]
  predictions <- [TO COMPLETE]
  mse <- [TO COMPLETE]
  return(mse)
}
```

1.5.3 Model selection based on estimated MSE (10 points)

- (5 points) Plot the validation MSE vs. `df` curve for each education group. Label axes and provide a title.
- (5 points) Which `df` value minimizes the validation MSE for each education group? Report these `df` values.

1.5.4 Fit chosen model and assessment (5 points)

- For each education group, fit a spline model on the entire training set using the `df` value selected above. Compute and report the test MSE on the corresponding test set.

1.6 Cross validation model selection (18 points)

1.6.1 Model selection (11 points)

- (5 points) Using the provided `cross_validate_spline()` helper, run 10-fold CV on the training set for each group over `df = 2:20`. Set the random seed as `set.seed(4710)`. Print the `cv_plot` for each group.
- (2 points) From the CV results, report the following for each group:
  - df.min (df minimizing CV error)
  - df.1se (simplest df within 1 SE of the min)
- (2 points) Based on the CV plots, which df value would you choose for each group?
- (2 points) Comparing the df values chosen by the validation set approach and by CV, do they agree? If not, which df value would you choose based on the intuitions on the MSE figures for each group?

1.6.2 Final fit (5 points)

- (3 points) For each group, fit a spline model on the entire training set using the `df` value selected by cross validation (either df_min or df_1se, and say which). Evaluate the *training MSE* and *test MSE* for each.
- (2 points) How do the training and test errors compare? What does this suggest about the extent of overfitting that has occurred?

1.6.3 Compare with validation set approach (2 points)

- (2 points) Compare the test MSE values from the CV approach to those obtained by the validation set approach, for both education groups.

2 Case study: kNN classification with Default data (36 points)

In this case study, you will use the `Default` data from the `ISLR` package to predict whether an individual defaults on their credit card payment. The data will be in the `Default` tibble.

```r
library(ISLR2)
data("Default")
```

2.1 Data split (2 points)

- Use the code like in Section 1 to split the `Default` data into a training set (80%) and a test set (20%), storing these in tibbles called `default_train` and `default_test`, respectively. Set the random seed as `set.seed(4710)`.

2.2 Class imbalance and baseline (6 points)

- (2 points) Compute and report the proportions of `Yes` and `No` in the `default` column of the training set. What did you notice?
- (2 points) Implement a baseline classifier that always predicts the most common class in the training set. Compute and report the *accuracy* of this baseline classifier on the test set.
- (2 points) Is this baseline classifier a good one?

2.3 kNN classification (28 points)

We have run the kNN classification algorithm for you for $K = 1, 2, ..., 20$, with a training and test split of 80% and 20%, together with the validation set approach within the training set. The results have been saved in `knn_counts_by_K.csv`. The file contains the confusion matrix counts (TP, FP, TN, FN) for the learning, validation, and test sets for each k value.

2.3.1 Load results

- Read in the `knn_counts_by_K.csv` file and store it in a tibble called `knn_counts`.

2.3.2 Compute error metrics in classification (4 points)

- Using `knn_counts`, compute the accuracy, precision, recall, and F1-score for all of the learning, validation, and test sets for each K value. Add these columns named `accuracy`, `precision`, `recall`, and `F1_score` to the tibble `knn_counts`. Print the tibble.

2.3.3 Plot metrics vs. K (6 points)

- (2 points) Create a lineplot of accuracy versus K for the learning and validation sets. Color the two lineplots by the `split` variable. Label axes and provide a title.
- (2 points) Create a lineplot of recall versus K for each of the learning and validation sets. Color the two lineplots by the `split` variable. Label axes and provide a title.
- (2 point) What do you notice about the accuracy and recall values? Using the formulas for computing accuracy and recall, which of the four confusion matrix counts (TP, FP, TN, FN) do you think are contributing to the issue?

2.3.4 Model selection and improvement (conceptual) (8 points)

- (1 points) Suppose you are a risk manager in a bank, and you think the cost of failing to identify a default is pretty high. Between accuracy and recall, which metric would you use to judge the performance of the classifiers and conduct model selection?
- (2 points) Is it a good idea to use this metric and the corresponding plot to select a value of K (using the validation set approach)? Why?
- (2 points) The kNN algorithm uses majority vote to classify a new observation. What issue it might have in this case?
- (2 points) How might you improve the algorithm to address the above issue?

2.3.5 Model complexity (conceptual) (10 points)

- (3 points) What happens to the model complexity as K increases? Why?
- (3 points) The degrees of freedom for KNN is sometimes considered n/K, where n is the training set size. Why might this be the case?
- (2 points) Conceptually, why might increasing K tend to improve the prediction rule? What does this have to do with the bias-variance tradeoff?
- (2 points) Conceptually, why might increasing K tend to worsen the prediction rule? What does this have to do with the bias-variance tradeoff?